

RNA_parte_1

YofreH

2025-09-19

Tipos de capas en TensorFlow/Keras,

Las capas (layers) son los **bloques de construcción** de una red neuronal. **Guía organizada** con los tipos de capas más usados, su función y en qué casos se recomiendan.

1. Capas densas (fully connected)

```
tf.keras.layers.Dense(units, activation='relu')
```

- **Qué hace:** conecta cada neurona de entrada con todas las de salida (clásico perceptrón multicapa).
- **Usos recomendados:**
 - Clasificación o regresión con datos tabulares.
 - Capas finales en CNNs o RNNs para tomar la decisión.

2. Capas de convolución

```
tf.keras.layers.Conv2D(filters, kernel_size, activation='relu')
tf.keras.layers.Conv1D(...)
tf.keras.layers.Conv3D(...)
```

- **Qué hacen:** detectan patrones locales mediante filtros (ventanas deslizantes).
- **Usos recomendados:**
 - **Conv2D** → imágenes.
 - **Conv1D** → series temporales, audio, texto secuencial.
 - **Conv3D** → videos o volúmenes 3D (p.ej. imágenes médicas).

3. Capas de pooling (reducción de dimensión)

```
tf.keras.layers.MaxPooling2D(pool_size=2)
tf.keras.layers.AveragePooling2D(...)
tf.keras.layers.GlobalAveragePooling2D()
```

- **Qué hacen:** reducen el tamaño de la representación manteniendo información importante.
- **Usos recomendados:**
 - Evitar overfitting en CNNs.
 - Hacer redes más rápidas y compactas.

4. Capas recurrentes

```
tf.keras.layers.SimpleRNN(units)
tf.keras.layers.LSTM(units)
tf.keras.layers.GRU(units)
```

- **Qué hacen:** mantienen memoria sobre secuencias.
- **Usos recomendados:**
 - Procesamiento de texto secuencial.
 - Series temporales (predicciones de demanda, clima, etc.).
 - LSTM y GRU son preferidas porque resuelven mejor el problema del gradiente desvanecido que SimpleRNN.

5. Capas de embeddings

```
tf.keras.layers.Embedding(input_dim, output_dim)
```

- **Qué hace:** convierte categorías o palabras en vectores densos de baja dimensión.
- **Usos recomendados:**
 - NLP (representación de palabras, frases).
 - Datos categóricos con muchas clases (ej. códigos de producto).

6. Capas de normalización

```
tf.keras.layers.BatchNormalization()  
tf.keras.layers.LayerNormalization()
```

- **Qué hacen:** reescalan activaciones para estabilizar entrenamiento.
- **Usos recomendados:**
 - Redes profundas (CNNs, RNNs, Transformers).
 - Aceleran la convergencia y reducen sensibilidad a inicialización.

7. Capas de regularización

```
tf.keras.layers.Dropout(rate=0.5)  
tf.keras.layers.SpatialDropout2D(rate=0.2)
```

- **Qué hacen:** apagan neuronas aleatoriamente durante entrenamiento.
- **Usos recomendados:**
 - Cuando hay riesgo de **overfitting**.
 - Dropout clásico en densas, SpatialDropout en CNNs.

8. Capas de activación

```
tf.keras.layers.Activation('relu')
```

- **Qué hacen:** aplican funciones no lineales (ReLU, sigmoid, tanh, softmax...).
- **Usos recomendados:**
 - ReLU → capas ocultas.
 - Sigmoid → clasificación binaria.
 - Softmax → clasificación multiclase.
 - Tanh → algunas RNNs.

9. Capas especiales de TensorFlow

- **Attention / MultiHeadAttention:**

```
tf.keras.layers.MultiHeadAttention(num_heads=8, key_dim=64)
```

→ Base de los Transformers (NLP, visión).

- **Reshape, Flatten, Concatenate:** manipulan la forma de los tensores.
- **ConvLSTM2D:** combina convolución y memoria → videos, secuencias de imágenes.

10. Capas de salida típicas

- **Clasificación binaria:** Dense(1, activation='sigmoid')
- **Clasificación multiclase:** Dense(n_clases, activation='softmax')
- **Regresión:** Dense(1, activation=None)

¡Excelente! Vamos a profundizar en **la capa de convolución 2D (Conv2D)**, que es fundamental en redes neuronales convolucionales (CNNs). Te doy tres cosas: **explicación detallada, dibujo conceptual y ejemplo interpretativo (sin código)**.

2. ¿Qué es una capa Conv2D?

Es una capa que aplica **filtros (kernels)** sobre una entrada bidimensional (imagen, mapa de características, etc.) para detectar patrones locales. Cada filtro es una **ventana deslizante** que se mueve sobre la entrada multiplicando y sumando valores → genera un **mapa de activación**.

2.2. Forma y parámetros principales

- **filters** → número de filtros (cuántos mapas de activación se generan).
- **kernel_size** → tamaño de cada filtro (ej. 3×3, 5×5).
- **strides** → cuántos píxeles se mueve el filtro cada vez.
- **padding** → si se rellenan los bordes ("same") o no ("valid").
- **activation** → función no lineal aplicada al resultado (ej. ReLU).

2.3. Dibujo conceptual

Imagina una **imagen de entrada** de 5×5 píxeles en escala de grises (un solo canal):

```
Entrada (5x5)
[ 1  2  3  0  1 ]
[ 0  1  2  3  1 ]
[ 3  2  1  0  2 ]
[ 2  0  1  2  3 ]
[ 1  1  0  2  1 ]
```

Y un **filtro 3×3** (ejemplo detecta bordes):

```
Kernel (3x3)
[-1  0  1]
[-1  0  1]
[-1  0  1]
```

Cuando el filtro se "desliza" sobre la imagen:

- Se multiplica elemento a elemento (convolución).
- Se suman los resultados.
- Se obtiene un **nuevo valor** en la salida.

2.4. Ejemplo interpretativo (sin programar)

Supón que usamos:

```
Conv2D(filters=2, kernel_size=(3,3), strides=(1,1), padding="valid", activation="relu")
```

a) Entrada

Una foto en escala de grises de 28×28 (como MNIST).

b) Proceso

- **filters=2** → usaremos 2 filtros distintos (p. ej. uno detecta bordes verticales, otro detecta bordes horizontales).
- **kernel_size=(3,3)** → cada filtro observa "ventanas" de 3×3 píxeles.
- **strides=(1,1)** → el filtro se mueve 1 píxel cada vez (máxima resolución).
- **padding="valid"** → no se agregan bordes, así que la salida es de 26×26 (más pequeña que la entrada).
- **activation="relu"** → convertimos los valores negativos en 0, conservando solo activaciones fuertes.

c) Salida

- Cada filtro genera un mapa de activación de 26×26.
- Con 2 filtros, obtenemos una salida de **26×26×2**.

Interpretación:

- El primer mapa ilumina dónde hay bordes verticales en la imagen.
- El segundo mapa ilumina dónde hay bordes horizontales.
- Estos mapas se usan como **entrada a capas posteriores**, que combinan patrones básicos en estructuras más complejas (esquinas, formas, objetos).

2.5. Analogía intuitiva para comprender su uso

- Piensa en la imagen como un **texto escrito en papel**.
- Los filtros son como **marcadores transparentes**: uno que resalta solo las **líneas horizontales**, otro solo las **verticales**.
- Al pasar estos filtros por el texto, obtienes “mapas” que muestran solo esas características.
- Cuantas más capas añades, más complejo es lo que detectas (bordes → formas → dígitos → rostros).

Mini-ejemplo numérico paso a paso de una convolución 2D.

1. Entrada

Imaginemos una **imagen 4×4 en escala de grises** (cada número es la intensidad de un píxel):

$$X = \begin{bmatrix} 1 & 2 & 0 & 1 \\ 3 & 1 & 2 & 2 \\ 0 & 1 & 3 & 1 \\ 2 & 0 & 1 & 1 \end{bmatrix}$$

2. Filtro (kernel)

Usaremos un **filtro 2×2** que actúa como **detector de bordes**:

$$K = \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix}$$

3. Proceso de convolución

- **Stride = (1,1)** → el filtro se mueve de 1 en 1.
- **Padding = valid** → no se agregan bordes, así que la salida será de tamaño $((4-2+1) \times (4-2+1) = 3 \times 3)$.

Para cada posición:

$$(X \odot K)_{i,j} = \sum_{u,v} X_{i+u,j+v} \cdot K_{u,v}$$

4. Cálculos paso a paso

(a) Primera ventana (esquina superior izquierda):

$$\begin{bmatrix} 1 & 2 \\ 3 & 1 \end{bmatrix} \odot \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix} = (1 \cdot 1) + (2 \cdot -1) + (3 \cdot 1) + (1 \cdot -1)$$

$$= 1 - 2 + 3 - 1 = 1$$

(b) Segunda ventana (fila 1, columna 2):

$$\begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix} \odot \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix} = (2 \cdot 1) + (0 \cdot -1) + (1 \cdot 1) + (2 \cdot -1)$$

$$= 2 + 0 + 1 - 2 = 1$$

(c) Tercera ventana (fila 1, columna 3):

$$\begin{bmatrix} 0 & 1 \\ 2 & 2 \end{bmatrix} \odot \begin{bmatrix} 1 & -1 \\ 1 & -1 \end{bmatrix} = (0 \cdot 1) + (1 \cdot -1) + (2 \cdot 1) + (2 \cdot -1)$$

$$= 0 - 1 + 2 - 2 = -1$$

5. Resultado completo

Haciendo lo mismo para todas las posiciones, la **salida (feature map) de 3×3** es:

$$Y = \begin{bmatrix} 1 & 1 & -1 \\ -1 & 1 & 1 \\ 1 & -2 & 1 \end{bmatrix}$$

6. Interpretación

- Los valores positivos indican coincidencias fuertes con el patrón del kernel (bordes en cierta dirección).
- Los valores negativos indican la presencia del patrón contrario.
- Si aplicamos una **ReLU**, los negativos se vuelven 0, quedando solo las activaciones relevantes.

RNA: Perceptrón Simple

Objetivo

- Entender **qué es** un perceptrón simple.
- Visualizar su **ecuación** y su **interpretación geométrica**.
- Ver **cuándo funciona** y **cuándo falla** (AND vs XOR).
- Conectar posteriormente con redes más profundas → Conv2D.

¿Qué es el perceptrón?

El **perceptrón** (Rosenblatt, 1958) es el modelo más simple de neurona artificial.

$$y = f\Big(\underbrace{\sum_{i=1}^n w_i x_i}_{\text{combinación lineal}} + b\Big)$$

- x_i : características de entrada
- w_i : pesos (importancia)
- b : sesgo (umbral)
- f : activación (p.ej., *step*, *sigmoid*, ReLU)

Idea: aprende una **frontera lineal** que separa dos clases.

Intuición geométrica

- En 2D aprende una **recta**; en 3D, un **plano**; en general, un **hiperplano**.
- Clasifica según el **lado** de la frontera donde cae el punto.

$$\underbrace{w_1 x_1 + w_2 x_2 + b = 0}_{\text{frontera de decisión}}$$

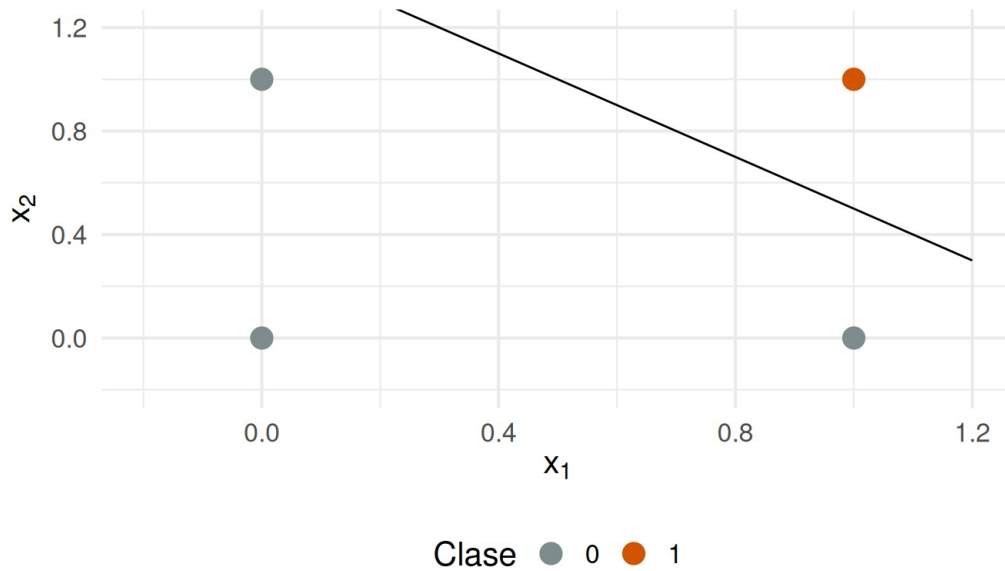
Ejemplo: compuerta AND

Tabla de verdad:

x_1	x_2	y
0	0	0
0	1	0
1	0	0
1	1	1

El perceptrón **sí** puede separar AND con una recta.

Perceptrón en AND: frontera lineal ilustrativa



¿Por qué la activación?

Sin activación, el modelo es **lineal** de extremo a extremo. Con activación (sigmoid/ReLU), la neurona puede **aprender umbrales** y producir **probabilidades**:

- $\text{sigmoid}(z) = 1/(1 + e^{-z}) \rightarrow$ salida en (0,1).
- $\text{ReLU}(z) = \max(0, z) \rightarrow$ útil en redes profundas.

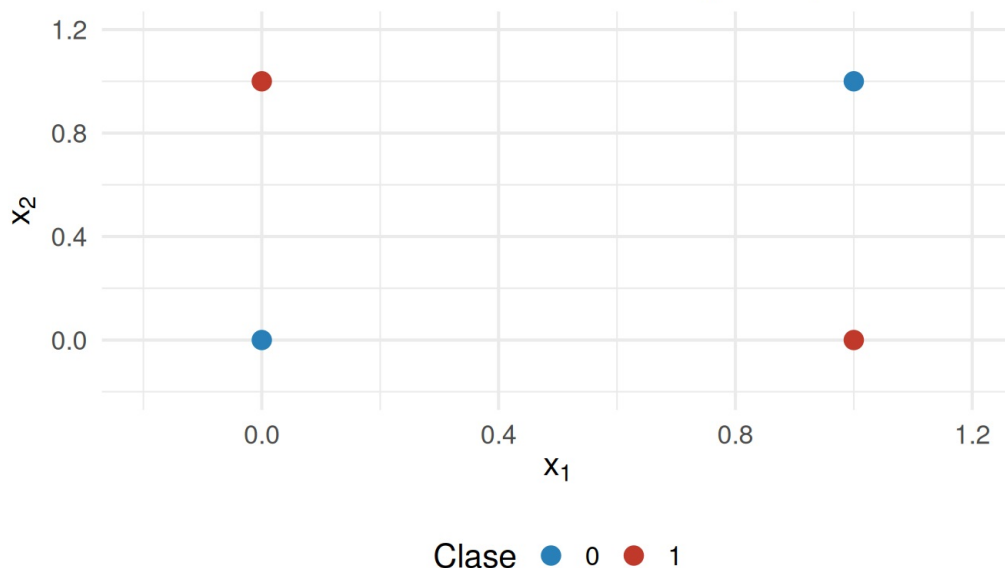
¿Cuándo falla? XOR

Tabla XOR:

$\backslash(x_1)$	$\backslash(x_2)$	$\backslash(y)$
0	0	0
0	1	1
1	0	1
1	1	0

XOR **no es linealmente separable** → un perceptrón simple **no puede** resolverlo.

XOR: no existe una frontera lineal que separe las clas



Componentes y efecto en la frontera

- **Pesos $\backslash(w\backslash)$** : rotan/inclinan la frontera.
- **Sesgo $\backslash(b\backslash)$** : traslada la frontera.
- **Tasa de aprendizaje** (en entrenamiento): qué tan rápido cambian $\backslash(w,b\backslash)$.
- **Pérdida** (p.ej., *binary cross-entropy*): mide el error de clasificación probabilística.

Conexión con el taller (Colab)

1. Entrenar un perceptrón en **AND** (Bloque 1).
2. Visualizar **pesos y fronteras**.
3. Notar la limitación: **XOR**.
4. **Próximo bloque: MLP** (capas densas adicionales) → combina neuronas para fronteras no lineales.
5. Luego, datos de **imágenes** y **Conv2D**.

Cierre

- El perceptrón es la **unidad elemental**.
- Su límite (separación lineal) **motiva** redes más profundas.
- Esta base conceptual es clave antes de entrar a **CNNs (Conv2D)**.

Referencias breves

- Rosenblatt, F. (1958). *The perceptron: A probabilistic model for information storage and organization in the brain*.
- Goodfellow, I., Bengio, Y., Courville, A. (2016). *Deep Learning*. MIT Press.
- Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.

Excelente observación — sí: el entrenamiento de una red neuronal puede formularse como un **problema de optimización sin restricciones**, pero con una estructura muy específica. Vamos a **reformular el planteamiento general** con todas las variables definidas y ejemplos concretos de la función de pérdida.

Planteamiento matemático general

El entrenamiento de una red neuronal (RNA) consiste en **encontrar los parámetros** que minimicen la pérdida promedio sobre un conjunto de datos de entrenamiento. Formalmente:

[]

* Definición de cada símbolo

Símbolo	Significado matemático	Interpretación en una RNA
(W^d)	Vector de parámetros (dimensión (d))	Todos los pesos y sesgos de las capas de la red
(x_i^p)	Vector de características de entrada	Ejemplo de entrada (imagen, vector numérico, texto codificado)
(y_i)	Etiqueta o salida deseada	Clase, valor o distribución objetivo
$(f_{W:p})$	Función implementada por la red con parámetros (W)	El modelo ; composición de capas no lineales
$(\cdot_+)$	Función de pérdida individual	Mide el error entre predicción y valor real
(n)	Tamaño del conjunto de entrenamiento	Número total de ejemplos usados

¿Por qué es un problema sin restricciones?

No se imponen condiciones explícitas como $(g(W)=0)$ o $(h(W))$; el objetivo es simplemente minimizar $((W))$ sobre todo el espacio de parámetros.

Sin embargo, sí existen **restricciones implícitas**:

- Los pesos (W) están determinados por las **dimensiones de las capas**.
 - Las **funciones de activación** (ReLU, Sigmoid, etc.) introducen **no linealidad**.
 - En la práctica, se usa **regularización** para evitar soluciones no deseadas (esto se incorpora al funcional, no como restricción).
- Por tanto, el problema es **sin restricciones explícitas**, pero **estructuralmente no convexo** y de **alta dimensión**.

Ejemplos comunes de funciones de pérdida ()

1. Regresión (salida real)

$$J(f_W(x_i), y_i) = \frac{1}{2} \|f_W(x_i) - y_i\|_2^2$$

Mide el error cuadrático medio (MSE). Ejemplo: predecir temperatura, precio o valor continuo.

2. Clasificación binaria (salida en $\{0, 1\}$)

$$J(f_W(x_i), y_i) = -\sum_{k=1}^K y_{ik} \log(f_W(x_i)_k)$$

Es la **entropía cruzada binaria**, usada en redes con activación **sigmoide** en la salida.

3. Clasificación multiclase (salida softmax)

$$J(f_W(x_i), y_i) = -\sum_{k=1}^K y_{ik} \log(f_W(x_i)_k)$$

Donde (K) es el número de clases y y_{ik} es 1 si el ejemplo pertenece a la clase (k). Usada en redes con **softmax** en la salida (p. ej. MNIST).

4. Con regularización L2 incorporada

$$J(W) = \frac{1}{n} \sum_{i=1}^n J(f_W(x_i), y_i) + \frac{\lambda}{2} \|W\|_2^2$$

Aquí ($\lambda > 0$) controla la penalización de pesos grandes. En Keras:
`kernel_regularizer=regularizers.l2( $\lambda$ )`.

Naturaleza del problema

- **No convexo:** la composición de activaciones no lineales crea múltiples mínimos locales.
- **Diferenciable casi en todo punto:** permite el uso de métodos de gradiente (SGD, Adam, RMSprop).
- **De gran dimensión:** (d) puede ser (10^5) a (10^8) .
- **Estocástico:** el gradiente se aproxima con minibatches.

Analogía geométrica

En el espacio de parámetros (W), la función $J(W)$ define una “superficie de error”. El método de descenso de gradiente estocástico (SGD) sigue una **trayectoria ruidosa descendente** sobre esa superficie hasta llegar a una **región plana de mínima pérdida promedio**.